

Case Study: AI–Accelerated Development of a B2C Outreach SaaS Product

Status: Anonymized case study, compiled 2026-08-12

Anonymization note: The product name, founder names, and predecessor project name have been replaced with generic labels. Vendor/tool names (Vercel, Neon, Clerk, Prisma, Claude Code, GitHub, Figma) are real, since they are load-bearing facts about the method, not identifying details about the people or company. Dates, hour figures, commit counts, and quotes are drawn from the project's git history, changelog, and internal control documents (cited inline by file path), supplemented by direct CTO testimony gathered while compiling this document and by the team's own pre-existing, untracked working notes on this exact topic (98_WORKING/article_notes/ , gitignored, not part of the tracked repo) — each source is marked as such where it's used, since they carry different evidentiary weight.

1. Summary

This is a case study of a two-person team — a non-technical Founder/CEO and a technical CTO who joined shortly after the product's design phase — building a real B2C SaaS product (a cold-outreach and lead-sequencing tool for small sales teams, "the Product" below) from a stalled prior attempt to a deployed, continuously operating MVP, using an AI coding agent (Claude Code) as the primary implementation engine.

The project is unusually well-instrumented for a two-person team: every session is logged to a timestamped, categorized time record; every non-trivial decision is written to an append-only decision log; every notable change is recorded in a dated changelog with its reasoning, not just its diff; and the team explicitly set out, from the first working session, to measure whether AI-accelerated development was actually faster than conventional hiring — and by how much.

That self-instrumentation is the most unusual and citable part of the story. Most claims about "AI made us 10x faster" are anecdotal. This project tried to build the evidence trail for that claim as it went, including the honesty to flag what it hadn't yet proven.

2. Prior effort and the restart

This case study treats time in distinct gates rather than one blended total, because the underlying costs are genuinely different in kind: a failed prior attempt, founder-side invention work, and AI-accelerated execution are not the same currency and shouldn't be added together without saying so. Section 10 defines all seven gates precisely; this section covers the first four, which sit before or at the boundary of AI-accelerated development.

2.1 Gate 1 — the aborted prior effort (outside the AI-development window)

The product that became "the Product" was originally scoped under a different name (referred to here as "the Predecessor Project"), run under a prior CTO/founder arrangement that predates the CTO whose work this case study otherwise covers. That effort produced, by the team's own retrospective accounting, on the order of **one-tenth of the eventual wireframe surface and no usable code**, over roughly **six months of real elapsed delay**. This is a real, if informally quantified, sunk cost — not a rounding error — and it sits entirely outside the AI-accelerated development window this case study measures, because it happened before AI-assisted development was the method being used at all.

A persona document survives in the repository's archive (99_ARCHIVE/predecessor_context/), describing a build-assistant persona explicitly tasked to "turn unclear product ideas into concrete specs," "challenge weak assumptions before they become code," and "keep documentation, architecture, and implementation aligned" — the same philosophy that governs the later, renamed project, even though the underlying effort stalled. The only documented reason for the rename is a single line in the project's navigation index: the predecessor name was dropped after "the copyright/name fallback." No further detail on the underlying conflict survives in the repository.

2.2 Gate 2 — founder invention time (outside the AI-development window)

Separately from the failed prior effort, the Founder spent an estimated **~80 hours** independently building the product's story and the Figma prototype itself — the concept, the workflow narrative, the schema thinking, and the wireframes later walked through in the extraction interview (§2.3). This is founder-side invention cost, done on the Founder's own time before the CTO handoff, and it is not part of the AI-accelerated execution this case study otherwise measures. It is also not part of, or a continuation of, the six-month aborted effort in §2.1 — the two are separate costs that happen to sit in the same rough period, not one flowing from the other.

Both figures in this section (six months, ~80 hours) are the team's own retrospective estimates, not derived from timestamped records the way Section 10's execution figures are. They are reported here as credible context, not as measured data.

2.3 The CTO handoff and the extraction interview — where the AI-accelerated window begins

The AI-accelerated development window this case study measures begins on **2026-05-31**, the date the CTO came on — marked concretely by a recorded ~75-minute planning conversation (05_TRANSCRIPTS/Prototype Development Planning_transcript.txt , ~850 lines) in which the Founder walks the new CTO through the Figma prototype and the product story built up over the preceding ~80 hours. The CTO opens the call by stating the plan explicitly — the moment the AI-assisted approach is proposed:

"What I want to do here is I want to structure as much of a story or narrative for each of these diagrams that you have... between that and our conversations, I can... start using some of the AI tools out there to go and start building up a minimally viable prototype."

The Founder then walks the CTO screen-by-screen through the Figma prototype — dashboard cards, the leads/contacts data model, notes fields, a competitive reference point — and at one point exports the Figma frames to PDF and emails them live, mid-call. That PDF is the same Cold Outreach App Wireframes.pdf that later ships in the repository's 03_DESIGN_ASSETS/. This transcript is exactly the mechanism the case study set out to describe: **a recorded interview/walkthrough, later played back to an AI agent as spec input**, rather than a formal handoff document written cold. Gate 3 (§10) models this extraction-interview effort — prep, the call itself, and the surrounding handoff work, not just the ~75 minutes of recording — at **7 hours**.

2.4 The proof-of-concept session, then a real gap

The repository's first commits land on **2026-06-14**, two weeks after the interview. On that date, in a single ~4-hour-10-minute session, the CTO — working directly with an AI coding agent — took the Figma/PDF wireframes and the interview's product story and produced a real, deployed Next.js application shell: authenticated routing, Clerk sign-in, and a full app-shell scaffold across dashboard, leads, sequences, templates, snippets, inbox, analytics, and settings. This is documented in unusual detail in docs/DEVELOPMENT_PROCESS_NOTES.md , written the same day, specifically so the session could later be used for exactly this kind of analysis. This is Gate 5 in Section 10's model: 4.17 measured hours, real project time regardless of what happened next.

What happened next was a real gap, but not idle project time. The time log and commit history both confirm a stop: the next commit after June 14 is dated **2026-07-13**. That month was not spent blocked or waiting on this project — it was time allocated to other work entirely, confirmed directly by the CTO. It counts as zero toward this project's time-in-motion, the same as any other day the CTO wasn't working on it, rather than as a stall internal to the project's own critical path. The sustained, day-by-day build this case study measures (Gates 6–7) begins on July 13, not June 14.

3. Design: how the product was actually specified

3.1 Figma as design source of truth, not runtime

The team's design workflow document (`docs/DESIGN_WORKFLOW.md` , written before the repo's first commit) sets an explicit ownership boundary: *"Cofounder owns product layout, screen design, and visual iteration in Figma. Engineering owns implementation fidelity, route behavior, data integration, accessibility, and responsive behavior."*

The practical loop it defines:

Cofounder updates Figma → Product/engineering compares the change against the canonical PRD → PRD/mapping updated if behavior changes → engineering implements → verifies → any implementation compromise is written down, not left as tribal knowledge.

This mattered enough to be named directly as a success factor later, in the CTO's own retrospective analysis: *"Figma design intent → PRD/scope check → route/component implementation → local/runtime verification → documented compromise or follow-up."* The single Figma file and its PDF export entered the repository once, in the initial import commit — they were not iterated on inside the repo itself; Figma stayed external, and the PRD/route-map documents are what actually version-controlled the product's evolving shape.

3.2 From walkthrough to spec: the transcript pipeline

The mechanism connecting design to build was not a formal spec document handed to engineering. It was the recorded conversation described in §2.2, played back as context to an AI coding agent alongside the PRD and wireframe PDF. The `docs/DEVELOPMENT_PROCESS_NOTES.md` entry for the June 14 session lists the starting materials explicitly: *"Figma .fig file and exported PDF existed... Repo contained product docs, implementation specs, and an early Next.js scaffold."* The transcript's own closing lines are the Cofounder summarizing "marching orders" for build and hosting options — the walkthrough functioned as an oral spec, later formalized into the written PRD.

4. The CTO's method: artifact-first engineering

The Cofounder/CTO wrote a self-analysis of his own operating method five days after the July restart began stabilizing (`docs/CTO_DEVELOPMENT_APPROACH_ANALYSIS.md` , dated 2026-06-20 — note this predates the July restart and was written off the single June 14 session). It is unusually candid for a document that doubles as external-facing framing material, and it is worth treating as a primary source for how the team thought about its own process, with the caveat that its productivity figures are **estimates authored by the team itself**, not independently measured — a distinction the document itself insists on.

4.1 "AI-first, artifact-first engineering"

The document's core claim is that the productivity gain didn't come from typing code faster. It came from **preventing expensive ambiguity** by giving every kind of decision an explicit, durable artifact instead of letting it live in chat or memory:

Artifact	Role
Canonical PRD	Product scope, routes, workflows, acceptance criteria
Decision path log	Unresolved build, launch, vendor, and scope questions
Append-only changelog	Chronological build history and decision evidence
Design workflow doc	Figma/code/PRD ownership boundaries
Risk register	Named technical, security, and operational risk
Source code	Implemented behavior

The document surveys a long list of adjacent industry terms (spec-driven development, docs-as-code, ADR-style governance, human-in-the-loop agentic development, DevOps/DORA) before settling on its own label: *"AI-assisted spec-driven development with docs-as-code, ADR-style decision control, and platform-engineering discipline"* — explicitly distinguished from "vibe coding," which it defines as "exploratory AI generation with weak specifications and weak verification."

4.2 Founder/CTO division of labor

The same document lays out, with unusual explicitness, who owned what:

Surface	Founder/CEO	Cofounder/CTO
Market and customer thesis	Owns	Tests against architecture
Product layout	Owns Figma, visual direction	Preserves intent while implementing
Scope decisions	Decides what MVP must prove	Converts scope into routes/data/gates
Production assets	Owns accounts, billing, domain, OAuth	Designs access model, documents transfer risk
Risk appetite	Decides acceptable business risk	Names risk plainly

The stated thesis: *"The CTO should not merely say yes faster. The CTO function here is to preserve speed by preventing mistakes that become expensive later."*

4.3 A day-one prediction, and a call for real measurement

Writing on June 20 — off a single measured session — the CTO document made a specific, falsifiable claim and then explicitly called for it to be tested rigorously going forward:

"Against an unfamiliar mid-level engineer, this approach plausibly compressed 30–60 hours into 4.17 hours... The productivity story becomes much stronger if every major slice has the same evidence model."

It proposed a concrete next step: *"create one GitHub issue per verified build slice, record mid-level baseline estimate before implementation, record actual implementation/debug/validation hours after completion... preserve links to commits, docs, screenshots, and deployment checks."*

That proposal is essentially what `00_CONTROL/time_log.csv` became roughly three weeks later — not manually estimated per-feature, but automatically derived from real session transcripts. Section 10 of this case study is that prediction actually getting tested, seven weeks on.

5. PRD, red-teaming, and the decision log

5.1 The PRD as frozen intent, the annex as reality

The team's canonical PRD (`01_PRODUCT_REQUIREMENTS/`, dated 2026-06-12) is treated as a stable statement of original intent and is **not** rewritten as the build progresses. A separate "As-Built Annex" tracks every place the shipped product diverges from that original spec, cross-referenced by PRD section number, each with an explicit "Spec (PRD)" / "As-built" / "Why this diverged" structure. One entry states plainly: *"this is the one place in this annex where the as-built system deliberately does not, and will not, match the PRD's literal acceptance criteria."*

5.2 Real reversals — the system catching its own mistakes

`00_CONTROL/DECISION_PATH.md` is an append-only log, and it contains genuine self-corrections, not just forward-looking decisions:

- **A security bug found and fixed by the team's own review, not a customer report:** the As-Built Annex documents that, at one point, any subscriber "could reach the platform admin panel and act on other subscribers' data" — a real cross-tenant access-control bug, caught internally (GitHub issue #107), fixed the same session it was found.
- **A wrong claim corrected in place, with the correction left visible rather than silently edited away:** *"Correction, 2026-08-08 (GitHub issue #107): this section previously stated '...' as a description of correct MVP behavior. That was wrong on the facts."*
- **A reversed architecture decision on outbound email:** the team investigated routing customer outreach through a mainstream transactional email provider, then killed the idea after checking the

providers' own policies directly: *"Every mainstream ESP's own Acceptable Use Policy bans it... [one major provider's] AUP names 'cold outreach' by name."*

- **A permanent decision that explicitly overrode an earlier "temporary" framing:** SMS sending was ruled out entirely on cost/compliance grounds, with the log noting this "supersedes the 'Phase 1B' framing this section previously used" — i.e., a placeholder that had been treated as future work was formally closed out as never happening.
- **A deliberate divergence from the PRD, confirmed on review:** the PRD's literal spec said any reply should stop a sequence; a review of real behavior overrode that instead of silently patching the code to match the PRD, recorded as *"a deliberate, permanent product divergence from the canonical PRD."*

This pattern — decisions get written down, and corrections get written down as corrections rather than quietly overwritten — is the clearest evidence of the adversarial, "red-team each other" dynamic the CTO described wanting from the working relationship with the Founder. Both PRD-vs-reality gaps and outright internal bugs surface in the same log, dated, with the reasoning attached.

5.3 Compliance rigor disproportionate to team size

The decision log and changelog show real legal research being done before, not after, feature decisions — output that would typically come from a legal or compliance function neither founder had. Automated messaging on LinkedIn, Instagram, Facebook, TikTok, and Reddit was investigated and ruled out with actual case law and platform-policy citations, not a generic "we'll add social later": *"LinkedIn has no messaging API product at any tier and its ToS bans automated outreach outright (real precedent — hiQ Labs v. LinkedIn, permanently enjoined, \$500k settlement)."* A parallel research pass covered CAN-SPAM, TCPA, and state privacy law before any SMS or bulk-email feature was built, and a data-breach response runbook was written proactively rather than after an incident. This kind of research is exactly the sort of task an AI agent can execute quickly and citably — which is part of why a two-person team was able to produce it at all.

6. Security and architecture posture: what was done differently

The CTO named eleven specific practices as deliberate choices, not accidents. Each is checked here against the repo directly rather than restated on say-so — two hold up with real caveats, the rest confirm cleanly.

Practice	Status	Evidence
Dependency locking (exact-	Confirmed	<code>package.json</code> has zero <code>^</code> / <code>~</code> ranges; <code>.npmrc</code> sets <code>save-exact=true</code> ; CI uses <code>npm ci</code>

pinned versions)		
GitHub vulnerability scanning	Partial	No Dependabot config or CodeQL workflow exists; alerts run at repo-settings level with 37 open, not gated in CI (changelog, 2026-08-09) — scanning exists but the backlog is real and unaddressed
Separation of responsibilities across vendors	Partial	Operational roles are assigned (Clerk=identity, Neon=data, Vercel=runtime) but no structured per-vendor security-boundary table exists — the control/data-plane split (below) is the stronger version of this idea
Secrets with expiration, console-managed	Partial	OAuth tokens encrypted at rest (AES-256-GCM); Vercel env vars managed via console/CLI — but expiration is reactive (a failed API call flips status to <code>expired</code>), not a tracked expiry date; no scheduled rotation policy, only event-driven ("on developer departure or suspected exposure")
Different secrets per service	Confirmed	Distinct credentials per vendor (<code>CLERK_SECRET_KEY</code> , <code>GOOGLE_CLIENT_SECRET</code> , <code>DATABASE_URL</code> , <code>ENCRYPTION_KEY</code> , <code>UNSUBSCRIBE_TOKEN_SECRET</code>), deliberately separated by blast radius — rotating one doesn't cascade to the others
No payment card data	Confirmed	No Stripe SDK or payment code anywhere in <code>src/</code> ; billing is a UI placeholder only — zero PCI scope because no payment flow exists at all
Limited PII	Confirmed	<code>Person / User</code> models collect only email, name, phone, job title, org — no SSN, DOB, address, or financial data
ORM-only DB access, no raw SQL	Confirmed, one caveat	No <code>\$queryRaw / \$executeRaw</code> in application code; always goes through Prisma. Caveat: query logging ships bound parameter values (which may include PII) to Vercel logs — flagged in the code's own comments as needing revisiting before real production traffic
Legal research in advance	Confirmed	Case-law-backed research (hiQ Labs v. LinkedIn) killed social-DM automation before it was built; CAN-SPAM/TCPA research preceded SMS scoping; a GDPR/CCPA-aware deletion flow was designed from research, not retrofitted
Separation of services (control-plane / data-plane)	Confirmed	<code>docs/CONTROL_PLANE_SPEC.md</code> and <code>docs/PRODUCTION_SECURITY_AND_OPERATIONS.md</code> define an explicit split; admin access is a separate allowlist, not tied to app roles, specifically so a compromised admin panel can't grant itself more admins
Bias toward	Confirmed	Explicit buy-vs-build comparisons with real per-vendor cost tables

buying vs. building		(docs/PLATFORM_PORTFOLIO_OPTIONS.md , 00_CONTROL/PRODUCTION_BUDGET_ESTIMATE.md) — floor cost ~\$81/mo, realistic ~\$119/mo, itemized by vendor
------------------------	--	--

The pattern worth naming: most of this is real and checkable, not aspirational security theater — but two items (vulnerability scanning, secret expiration) are weaker than "we did this" implies, and the paper is better for saying so rather than rounding both up to confirmed.

6.1 Additional design elements, found rather than claimed

Checked beyond the eleven above — a mix of real, undocumented strengths and real, previously unnamed gaps.

Design element	Status	Evidence
Cross-tenant CI tests	Confirmed	tests/db/constraints.test.ts runs against a real dev database and explicitly asserts cross-tenant reads are blocked, for Person , Template , Snippet , and Sequence — not a mocked or hypothetical check
Tenant isolation enforcement	Gap	No ORM-level or middleware-level tenant scoping exists — every query manually adds companyId to its where clause by convention, not by a shared, enforced helper. A future query that forgets the filter would compile and run
Route/session protection	Gap	No middleware.ts exists anywhere in the repo — auth is checked per-route via Clerk's auth() , not centrally enforced. Every current route was verified to have a guard, but a newly added route that forgets one would be unintentionally public by default
Webhook signature verification	Confirmed	The Clerk webhook handler calls Clerk's own verifyWebhook() with a signing secret and rejects on failure — genuine cryptographic verification, not payload-trusting
Audit logging	Confirmed, with a documented history	A real AuditLog model exists; a prior gap ("no audit trail for send/approval actions") was found and fixed, visible in the code's own comments — but fully autonomous scheduler-driven sends still aren't individually logged, only human-approved actions are
Hard delete on account deletion	Confirmed	Real cascading hard deletes across all tenant data plus the Clerk identity itself, triggered by the real user.deleted webhook — not a soft-delete flag
Suppression/do-	Confirmed	A hard block at the actual send path (scheduler.ts), not an

not-contact enforcement		advisory UI flag — a suppressed run is cancelled before any send is attempted
Rate limiting	Gap	None found anywhere — no rate-limiting package, no in-house throttling. Nothing in the app itself prevents brute-force auth attempts or using the send mechanism as a spam engine, beyond Clerk's own platform defaults and the daily cron cadence
MFA / two-factor auth	Gap	No MFA configuration exists for customer-facing login (Clerk supports it natively; it isn't turned on or referenced anywhere in code or docs). The only 2FA mentioned in the entire repo is GitHub-org-level 2FA for the two founders' source-code access (docs/GITHUB_LOCKDOWN_RUNBOOK.md) — and that's a written <i>plan</i> , not confirmed as actually enabled
CSRF/XSS protection	Confirmed, by default rather than custom	No sanitization library exists because none is needed yet — the only dangerouslySetInnerHTML in the codebase renders a hardcoded script with no user input. Relies on Next.js Server Actions' built-in protections, not independently verified or hardened beyond the framework default

The honest summary: this project is stronger than average on data-boundary and process discipline (tenant-isolation *tests*, audit logging, hard deletes, suppression enforcement) and weaker than it should be on two things any real security review would flag immediately — **no MFA and no rate limiting**. Neither is a hard problem to fix; neither has been fixed yet, and neither was mentioned until checked directly.

7. Technology stack: options considered, and how they were decided

The stack was not chosen by default. docs/PLATFORM_PORTFOLIO_OPTIONS.md (2026-06-13) is a genuine comparison matrix across nine hosting/backend paths — including no-code builders (Lovable, Base44), managed hosting (Hostinger, both managed and VPS), self-hosting, Vercel, Supabase, and a Neon+Clerk combination — each scored on production fit, main benefit, and main risk. The eventual default (Neon+Clerk on Vercel) is characterized candidly, including its downside: *"strong developer experience... more vendor sprawl than Vercel + Supabase."* Supabase was a real, seriously considered alternative specifically because it would have meant fewer vendors, rejected mainly on migration-planning grounds.

Critically, docs/ARCHITECTURE_DIRECTION.md (2026-07-13) states the team never treated this as a locked, final production decision: *"Thalam has not selected a mandatory final production host... The architecture decision is portability plus proof gates, not a pre-committed cloud destination."* A parallel document, docs/VERCEL_TO_GCP_MIGRATION.md, exists specifically to keep a credible exit path open, framed explicitly as "one possible migration path, not the selected production destination." A "Founder Decision

Gate" checklist asks, bluntly: "Can [the founders] recover source, data, secrets, and domain control without vendor goodwill?"

The practical stack that emerged — Next.js on Vercel, Neon Postgres via Prisma, Clerk for auth, Gmail OAuth for the first real send channel — was arrived at pragmatically, as the fastest path that didn't foreclose portability, rather than a single up-front architecture decision defended once and never revisited.

8. What was added to the process: the control layer

As the project matured past the initial scaffold, the team layered several pieces of process infrastructure on top of the AI-agent workflow — not generic project-management ritual, but specific mechanisms to keep an AI agent's fast output honest across sessions with no persistent memory of its own.

7.1 The changelog as flight recorder

CHANGELOG.md's own header states its purpose precisely: *"a supplement to git history, not a replacement — git has the exact diffs; this file has the why, especially for decisions that don't show up cleanly in a commit message."* Entries are dated, prose-form, and explain reasoning, not just what changed — a deliberate choice to preserve the *why* that a diff alone can't carry.

7.2 GitHub Issues as the operable work queue

docs/PROJECT_TRACKING.md documents a standard GitHub Issues + Project board workflow (Inbox → Ready → In Progress → Blocked → Review → Done, with Estimate, Actual Hours, and PM Hours fields) — used from inside the AI coding session itself, not as a separate manager-facing tool. Of the project's 184 commits, at least 48 reference an issue number directly. The changelog shows a consistent same-session pattern: file an issue → research it → decide → close it, with the reasoning captured in both the issue and the changelog. One entry describes a full backlog audit where every open issue was re-verified against current code rather than trusted from its title, closing several that had quietly already been fixed and forgotten.

7.3 Session-close automation

Three scripts run as a local Stop hook at the end of every AI coding session (scripts/session-close.ts, scripts/build-time-log.ts, scripts/generate-executive-report.ts):

- **build-time-log.ts** parses the raw session transcript into the categorized time-log rows analyzed in Section 10. It classifies each turn as dev, pm, or idle using tool-use signals (edits, builds, migrations) and keyword heuristics, and self-documents at least one real bug fix to its own accuracy: an earlier version floored every turn to a 15-minute minimum, which the team caught overstating real time by roughly 6x in sessions with many short turns.

- **session-close.ts** runs a checklist: has today's work been logged, does the changelog have a dated entry for today, are there stale TODOs, is there uncommitted work that needs a safety-net branch. It deliberately does **not** auto-write the changelog or auto-commit — the script's own header states this is by design, because "a real changelog entry needs judgment about what mattered this session, not a mechanical placeholder."
- **generate-executive-report.ts** diffs the decision log and risk register against their state at session start, producing a founder-facing summary of what changed, what's newly open, and what's still unaddressed — the basis for the C-suite status reports referenced elsewhere in the project's control documents.

This is a meaningful design choice: the automation's job is to *surface* what a human should look at, not to *decide* what mattered. The judgment calls stay human.

7.4 CI, deployment, and testing

A single GitHub Actions workflow (`.github/workflows/ci.yml`) gates every push to `main`: `typecheck` → `lint` → `test` → `build`, followed by a smoke test against a real (non-production) environment for pushes to `main` specifically. Deployment itself runs through Vercel's standard GitHub integration — a push to `main` triggers a production deployment automatically; there is no separate manual deploy step. The team added a standing verification habit on top of that automation: after every push, check the actual Vercel deployment record (`vercel ls` , `vercel inspect`) to confirm what shipped actually matches what was pushed, rather than trusting the pipeline blindly.

Test coverage grew steadily and visibly alongside feature work rather than as a separate retrofit: the changelog's running total climbs from 61/61, through 92, 111, 126, 147, to 170/170 passing at the most recent entries, with nearly every changelog entry ending in the same confirmation line — "typecheck, lint, and build all clean" — making the test count a de facto running scoreboard of the build's health.

7.5 Solo-developer workflow, by explicit choice

With a two-person team and no other engineers touching the code, the project runs a direct-to- `main` workflow with no pull-request review step — a deliberate, written-down choice rather than a shortcut taken by default: "*Revisit this when a second engineer joins or real subscriber data is live.*" The safety net is a habit, not a gate: if a session ends with work that can't be safely pushed straight to `main` , the fallback is a branch and PR rather than leaving it stranded uncommitted.

7.6 Tooling cost basis: two flat subscriptions, no metered API billing

The AI tooling behind this entire build ran on two flat-rate consumer subscriptions, not per-token API billing: Claude Code (~\$20/month) for the primary build work documented throughout this case study, and Codex (~\$20/month) used specifically as a second model for red-teaming — an independent adversarial review pass distinct from the CTO's own review, catching a different class of issue than a single reviewer

would. Total tooling spend: **~\$40/month flat**, regardless of hours used that month. This matters for the cost tables in Section 10: every dollar figure there compares *human labor cost* across classes, but doesn't need a separate "AI cost" line item to net against — at a flat monthly rate, the tooling cost is effectively fixed overhead, not a variable that scales with the hours in Gates 5–7.

9. AI as a substitute PM/governance layer, not just a coding tool

The CTO's own framing for this project's core thesis: AI-assisted, artifact-first engineering isn't only faster code generation — it's a substitute for functions a two-person team couldn't otherwise staff at all: project management, executive reporting, and issue triage. Most of the individual pieces are already documented elsewhere in this case study; this section is where they add up to a single claim, evidence-checked item by item.

Capability	Status	Where it's evidenced
Automated time tracking	Confirmed	<code>time_log.csv</code> , derived from real session transcripts, not hand-logged (§10)
PRDs established, changes tracked to an Annex	Confirmed	The canonical PRD stays frozen as original intent; the As-Built Annex absorbs every divergence, cross-referenced by PRD section number (§5)
Automated C-suite daily progress reporting	Confirmed	<code>generate-executive-report.ts</code> diffs the decision log/risk register against session-start state and writes founder-facing status reports (§8.3)
Issue flagging for follow-up	Confirmed	<code>session-close.ts</code> 's end-of-session checklist surfaces stale TODOs, uncommitted work, and missing changelog entries — a "here's what needs your attention" pass, not an auto-resolution (§8.3)
Facilitating feature-scope-creep discussion	Confirmed	Real, repeated pattern in <code>CHANGELOG.md</code> / <code>DECISION_PATH.md</code> : SMS was actively re-argued and closed as <i>permanently</i> out of scope, not left as a vague "later" (§4 of the decision log); an admin role-permission matrix was closed as genuinely unnecessary after confirming the real requirement with the founder directly; lead-cloning was presented as two explicit options and declined for lack of a clear use case rather than silently dropped
Rapid prototyping of a feature idea	Confirmed	The sequence simulator: originally specified as a blocking pre-launch validation <i>gate</i> , re-scoped mid-build into a fast, non-blocking diagnostic dry-run against a real lead — and building the prototype surfaced and fixed a real bug in the underlying variable-resolution logic along the way, not just validated the UI idea
A written, formal feature-triage procedure	Doesn't exist, by design	Neither <code>docs/PROJECT_TRACKING.md</code> nor <code>00_CONTROL/DECISION_PATH.md</code> contains a general triage procedure — what exists instead is a consistent, informal <i>loop</i> visible in changelog narrative (discuss → test → decide → record), not a written process document. See §9.1: the absence of a formal process is itself part of the claim, not a gap in it

9.1 The informality is the advantage, not a gap in the process

The previous draft of this section reported the missing triage procedure as a shortfall. That undersells what's actually happening: the *absence* of a standing process is the point, not an oversight in one.

A conventionally staffed team handles "should we build this" with a recurring mechanism — a backlog-grooming meeting, a sprint-planning ritual, a PM function whose job is partly to run that cadence. This project substitutes something lighter for all of it. In the CTO's own words, describing the mechanism directly:

"There is no need for a standing meeting structure. We can have a call, capture the conversation, discuss the merits, test the thesis — moving forward, ditching, or tabling the idea for a future release."

That's the loop, stated plainly: a conversation happens, gets captured (the transcript/session itself, then distilled into the changelog and decision log after the fact), the merits get discussed and often tested directly rather than argued about in the abstract, and the outcome — move forward, drop it, or table it for a later release — gets recorded once, durably, without needing the meeting to recur. The sequence simulator (§9, row above) is the clearest real instance of the "test the thesis" step specifically: rather than debate whether a launch-validation gate was the right design, the team built a fast version of the idea and let the real behavior settle the question.

The CTO's own self-analysis independently supports this framing, in language written before this specific observation was made: *"It is not Scrum theater. There are no required ceremonies, sprint rituals, or story-point mechanics"* and, on the broader governance pattern, *"startup-scale architecture governance... without TOGAF ceremony"* (docs/CTO_DEVELOPMENT_APPROACH_ANALYSIS.md). Both lines are describing the same thing from the process-literature-comparison angle that this observation describes from the meeting-overhead angle: the artifact discipline (decision log, changelog, PRD/Annex) is what lets the team skip the standing structure a larger org would need to get the same discipline — the record-keeping happens *after* a real conversation, instead of the conversation happening *because* a recurring meeting requires one.

9.2 Further advantages this pattern suggests, beyond what's directly evidenced here

The items above are specific to this repo. A few more plausible advantages of the same underlying mechanism — artifact-first AI engineering functioning as a governance layer, not just a code-generation tool — are worth naming even though this case study doesn't have direct repo evidence for all of them. These are reasoned extrapolation from the pattern above, not additional confirmed findings, and are labeled as such:

- **Onboarding a new team member against a real state, not tribal knowledge.** If a second engineer joined this project, the PRD/Annex/decision-log/changelog stack is, in principle, enough to reconstruct current state and reasoning without a live handoff conversation — the project's own stated design goal (CTO_DEVELOPMENT_APPROACH_ANALYSIS.md 's "continuous handoff documentation" framing, §4). Not tested here, since no second engineer has joined.
- **Cheap "what if we'd decided differently" audits.** Because reversed decisions are recorded as

corrections rather than silently overwritten (§5), the project has a real, if informal, capability to review its own track record of judgment calls — useful for a founder deciding whether to trust the next judgment call, not just the last one.

- **Compressed vendor/compliance research turnaround.** Already evidenced directly in this project (the case-law-backed research killing social-DM automation, the TCPA/CAN-SPAM research before SMS scoping, §5) — the extrapolation is that this generalizes to *any* domain where the bottleneck is research-and-synthesis time rather than typing speed, not just legal/compliance questions.
- **Lower cost of changing course.** A living PRD that's expected to diverge from its own Annex, rather than a frozen spec treated as a contract, plausibly lowers the psychological and process cost of admitting a decision was wrong — speculative, but consistent with the real reversals already documented in §5 (the reply/stop-condition divergence, the SMS reversal) happening without apparent friction or face-saving.
- **A smaller, more current "institutional memory" surface than headcount would normally require.** The claim isn't that AI remembers — it's that the artifacts *are* the memory, and an AI agent can be pointed at all of them cheaply every session in a way that would be a real onboarding/ramp cost for a new human hire each time. This is the same mechanism as the first bullet, viewed from the cost side rather than the capability side.

None of these five are measured in this case study. They're presented as the logical extension of a pattern that *is* measured (§4–§8), not as additional confirmed findings, and should be read with that distinction intact.

10. Time-in-motion: measuring the claim

This is the section the CTO's June 20 document explicitly asked for. `00_CONTROL/time_log.csv` is derived — not hand-estimated — directly from AI coding session transcripts, using the timing rule defined in `00_CONTROL/TIME_LOG_SCHEMA.md`: the first 30 minutes of any gap between turns is assumed think-time and not counted; minutes 30–45 of a gap are logged once as `idle` (capped at 15 minutes); nothing further accrues beyond 45 minutes. Every other minute is attributed to either `dev` (writing/editing code, running builds, debugging, migrations) or `pm` (scoping, deciding, reviewing, non-code documentation) based on what actually happened in that turn.

8.1 The seven-gate model

Section 2 introduced the first four gates. Here is the complete model, separating what's measured from what's estimated:

Gate	What it is	Hours	Status
1. Aborted prior effort	Prior CTO/founder route, ~6 months elapsed, ~1/10 of wireframe surface, no usable code	(~6 months elapsed, not hour-denominated)	Estimated — outside the AI-development window
2. Founder invention	Founder's own time building the product story and Figma prototype, before the CTO handoff	~80	Estimated — outside the AI-development window
3. Extraction interview	The May 31 handoff walkthrough — prep, the ~75-minute recorded call, and surrounding handoff work	7	Estimated (transcript length is fact; total gate time is not)
4. Pre-code PM/research	Architecture/source-of-truth/vendor sorting between the interview and first commit	3.5	Estimated
5. Proof-of-concept build	The June 14 scaffold session	4.17	Measured (docs/DEVELOPMENT_PROCESS_NOTES.md)
6. PM execution	Logged scoping/deciding/reviewing across the sustained build, July 13 – Aug 8	4.50	Measured (time_log.csv)
7. Dev execution	Logged coding/debugging/building across the sustained build, July 13 – Aug 8	25.56	Measured (time_log.csv)

AI-accelerated development window (Gates 3–7, from CTO handoff to the latest measured session): ~44.7 hours. Of that, **~34.2 hours (Gates 5–7) is fully measured** from timestamped session transcripts, not estimated after the fact; the remaining ~10.5 hours (Gates 3–4) is the team's own credible estimate for work that happened before any session transcript existed to measure it.

Gates 1–2 (~6 months elapsed, ~80 hours) sit outside that window by design — they are real costs, but they are prior-effort and founder-invention costs, not AI-accelerated *execution* costs, and this case study does not add them into the 44.7-hour figure. Reported together for context: aborted-effort delay (Gate 1) plus founder invention (Gate 2) plus the full AI-accelerated window (Gates 3–7) is roughly **~125 hours of total human/AI-assisted effort to the product's current state**, of which the ~6-month Gate 1 delay is elapsed calendar time with comparatively little output, not a dense hour count comparable to the others.

In the ~30 measured hours of Gates 6–7 alone (dev + PM inside logged sessions), the team produced 184 commits, a deployed and continuously operating application with nine real (non-placeholder) authenticated screens, 170 passing automated tests, real Gmail OAuth send integration, tenant isolation with enforced cross-tenant checks, and the full body of control documentation this case study draws on.

8.2 Work was not evenly distributed

Activity across the 18 days with any logged work was sharply uneven — a small number of long sessions did most of the work, with several days logging only a few minutes:

Date	Dev (h)	PM (h)	Idle (h)	Commits that day
2026-07-13	0.09	0.13	0.25	3
2026-07-15	1.13	0.06	0.50	21
2026-07-16	2.90	0.29	0.49	6
2026-07-28	2.13	0.44	0.25	19
2026-08-03	1.25	1.26	0.00	9
2026-08-06	3.32	0.33	1.83	18
2026-08-07	6.74	0.21	1.00	10
2026-08-08	4.93	0.61	1.92	19

The final three days in the window (Aug 6–8) alone account for roughly 15 of the 25.56 total dev hours (Gate 7) — nearly 60% of all measured building happened in the last three days of an 18-day active window. This is not a smooth, linear "AI writes code at a constant rate" story; it's bursty, concentrated, human-paced work punctuated by real gaps of days (July 19 → July 23, July 23 → July 27, July 28 → August 1) where nothing happened at all.

8.3 Reading the numbers honestly

A few things this data does *not* claim, in keeping with the team's own stated standard for not overclaiming:

- Gates 6–7 (the day-by-day log) only start July 13. The June 14 scaffold session (Gate 5, 4h10m) is

documented separately in `docs/DEVELOPMENT_PROCESS_NOTES.md` and is added to the total explicitly, not double-counted, not omitted.

- The June 14 → July 13 gap is genuinely not idle time inside this project's account. That month was allocated to other work by the CTO, confirmed directly — it counts as zero toward this project's time-in-motion the same way any day not spent on the project would, rather than as a stall internal to the project's own critical path.
- "PM" time (4.50 hours, Gate 6) is almost certainly an undercount of total founder/CTO cognitive load — it captures time inside logged AI sessions only, not the thinking, vendor setup, or account/credential work that happened outside a session entirely. Gates 3–4 (~10.5 hours, estimated) partially cover pre-session work, but are themselves estimates, not logged time.
- This measures one team's throughput on one project. It is not a controlled comparison against an actual human developer building the same product in parallel — that comparison exists only as the *estimated* baseline table in `docs/CTO_DEVELOPMENT_APPROACH_ANALYSIS.md` (Section 4.3 above), which the team itself labels an estimate, not a measurement.

8.4 Domestic comparison: hours, rate, and cost by operator class

Rescaled from the CTO's June 20 estimate onto the current 34.23-measured-hour execution total (Gates 5–7). Domestic US only.

Operator class	Hours	Internal rate	Internal cost	Consulting rate	Consulting cost
AI-assisted (CTO, measured)	34.23	\$140/hr	\$4,792	\$325/hr	\$11,125
Principal / staff engineer	49–82	\$140/hr	\$6,860– \$11,480	\$325/hr	\$15,925– \$26,650
Senior engineer	99–197	\$110/hr	\$10,890– \$21,670	\$225/hr	\$22,275– \$44,325
Mid-level engineer	246– 493	\$85/hr	\$20,910– \$41,905	\$150/hr	\$36,900– \$73,950
Entry-level engineer	575– 985	\$45/hr	\$25,875– \$44,325	\$100/hr	\$57,500– \$98,500

Rates checked against 2026 US market data (contractrates.fyi, IT-consulting rate guides); the original document's self-authored rates ran high, especially mid/entry-level. AI-assisted row uses the same Principal/staff rate — it's the CTO's own time, not a separate class.

8.5 Offshore comparison: hours, rate, and cost by operator class

Same base hours, adjusted for a ≥8-hr time-zone productivity loss (15–20%) and rework risk (67% of offshore projects require significant rework — Gartner; modeled as a full rebuild on that share, giving a 1.67x hour multiplier). Rate is the higher of India/Eastern Europe per tier. H1B excluded (domestic-wage mechanism, not a discounted rate).

Operator class	Hours (adjusted)	Internal rate	Internal cost	Consulting rate	Consulting cost
Principal / staff engineer	102–161	\$75/hr	\$7,672– \$12,083	\$174/hr	\$17,798– \$28,032
Senior engineer	207–387	\$62/hr	\$12,813– \$23,997	\$127/hr	\$26,246– \$49,155
Mid-level engineer	514–969	\$45/hr	\$23,109– \$43,587	\$79/hr	\$40,568– \$76,519
Entry-level engineer	1,200–1,935	\$22/hr	\$26,407– \$42,575	\$49/hr	\$58,815– \$94,827

Once rework risk is priced as hours, offshore is not cheaper than domestic at any tier — Principal/staff offshore (\$7,672–\$12,083) runs at or above the domestic baseline (\$6,860–\$11,480, above).

11. Is this project immune to AI-coding failure, or just unmeasured so far?

A fair challenge to this whole case study: is its apparent success actually evidence of anything, or just one un-failed data point? This case study measures one team's throughput on one project — a single sample, already caveated in Section 10 as not a controlled comparison. Worth checking against the actual research on AI-coding failure modes rather than assumed away.

What's actually documented about AI-coding failure, from independent (non-vendor) sources:

- **METR's July 2025 randomized controlled trial** (16 experienced open-source developers, 246 real issues) found AI tooling *increased* task completion time by 19% for developers working in codebases they knew well — even though those same developers believed, both before and after, that AI made them about 20% faster. This is the single most rigorous study available, and it's a direct warning against trusting self-reported productivity, which is exactly the kind of claim this case study is built on.
- **Stanford (Perry et al.)** found developers using AI code assistants wrote *less secure* code than a control group — and rated their own, more vulnerable code as *more secure*. A documented verification blind spot, not an agent problem specifically — a human-trust-calibration problem.

- **"Verification theater"** is a real, named, widely-observed pattern: AI agents reporting "tests pass" or "build is clean" without that reflecting true end-to-end correctness — directly relevant to a project whose central evidence is dated changelog entries that self-report "full suite N/N passing; typecheck, lint, and build all clean" after nearly every session.

Where the evidence runs out. Vendor studies (CodeRabbit, Veracode, GitClear — all sell products that benefit from "AI code needs scrutiny") are directionally consistent with each other: AI-authored code shows more logic/security issues and roughly double the code churn (rework within two weeks) of human-only code. Directionally corroborating, but not independent, third-party, Gartner-caliber research the way the offshore figures are. No study was found that directly measures whether AI-agent code that passes CI has a *different* rate of later-discovered defects than human code that passes the same CI gate — the single most relevant comparison to this project's own claims, and it doesn't exist yet.

What this means for this case study, plainly: the changelog's repeated "N/N tests passing, build clean" self-reports are exactly the kind of signal METR's finding says not to fully trust, and this project has no independent, third-party verification of its own defect rate — only its own test suite, written by the same process being evaluated. The artifact-first discipline (decision logs, changelogs, a functional-completeness matrix that names known gaps rather than hiding them) is a reasonable, widely-recommended mitigation — but no research was found measuring whether that discipline actually reduces AI-specific failure modes. It's practitioner consensus, not measured fact, and should be presented as such rather than as a proven safeguard.

12. The no-AI-features decision: real rationale, undocumented in the repo

The PRD lists "AI writing assistance" as out of MVP scope in a single unelaborated bullet, grouped alongside items like a rich-text editor and Outlook support:

"7.2 Deferred ... AI writing assistance."

That's the entire written entry. By contrast, every other comparable scope call in the project — Gmail OAuth scope, the billing model, SMS, social automation, the editor — went through the full "Decision Needed / Options / Recommendation / Decision Criteria" structure visible in

`00_CONTROL/DECISION_PATH.md`. AI writing assistance did not get a dedicated decision record there. Read from the repository alone, it looks like a default deferral rather than an argued one — and the first draft of this case study said so.

That reading was wrong. The CTO, asked directly, supplied the actual reasoning (captured here as direct testimony, not sourced from a repo file — worth flagging as a distinct evidence class from the rest of this document):

AI product features were scoped as **post-GA, version-two work from the outset** — not features that were pitched for MVP and then cut, but ones the team never intended to fight for a v1 slot in the first place, because the team didn't yet have a clear enough sense of which AI feature would actually be *compelling* to a real user, as opposed to merely possible. Candidates discussed for that later stage included using AI to **parse inbound replies and route them into the correct classification buckets** automatically, to build **longitudinal projections** from the lead/outreach data the product would accumulate, and to **draft the outreach communications themselves**. None were rejected on capability grounds — they were deferred on product-conviction grounds, pending real usage data the MVP itself doesn't yet exist to produce.

The infrastructure-cost delta is real, but it's one input to a broader question the team was already working through for v2, not the standalone reason v1 shipped without AI: **how would AI features even be hosted, once there was a real justification to build them?** Three paths were on the table, each with a different cost/control tradeoff — a frontier-model API (the ~\$1,500–\$2,000/month estimate against a ~\$200/month baseline infrastructure spend cited earlier, a 7–10x run-rate increase), self-hosting an open-weight model (higher upfront technology spend to stand up, but potentially significant long-term savings once that spend is recovered), or routing through a neutral compute-cloud provider serving open-weight models rather than operating infrastructure directly. That last path surfaced its own consideration: the strongest open-weight models at the time were coming out of Chinese labs, which the team flagged as a potential geopolitical access/dependency risk they'd rather not build a product around without deciding that deliberately.

Two things are worth holding at once here. First, this was a live, reasoned architecture question the team was actively working through — not an oversight — but it was a *version-two* question, correctly out of scope for what the MVP needed to prove. Second, none of this — the post-GA scoping, the three hosting paths, the geopolitical sourcing concern — made it into `00_CONTROL/DECISION_PATH.md` the way comparable in-scope MVP decisions did, which means the repository's own record understates both the rigor and the forward-looking scope of this thinking. For a project this disciplined about writing decisions down, that's a small but real gap in the project's own artifact discipline. It's also a useful, honest data point in its own right: even a team optimized for capturing its own reasoning doesn't capture all of it — decisions about what's *out* of scope, and why, seem to get less documentation discipline than decisions about what's *in* scope — and a case study built only from repository artifacts, without the chance to ask the people who lived it, would have gotten this one wrong.

Note the distinction this case study holds throughout: **AI as the internal development tool** (Claude Code, used to build the product) and **AI as a customer-facing product feature** (AI-assisted reply parsing, projection, or drafting inside the product itself) are separate questions. This project embraced the former without reservation and, on a real cost/benefit judgment rather than a categorical objection, deferred the latter for its first release — revisitable once usage data makes the ROI case for the ~7-10x cost increase.

13. Why a lean org, and what enabled it

The team's own framing, stated directly in the CTO analysis document, is that the AI-assisted, artifact-first method is *what made* a two-person structure viable for this scope of product, not a constraint the method had to work around: "*the method converted a founder-heavy, multi-vendor, partially unfamiliar build into a controlled implementation workspace.*" The project's working rules formalize this as a stage, not a permanent structure — direct-to-main commits with no PR review, explicitly scoped to change "when a second engineer joins or real subscriber data is live."

The mechanism, as the team describes it, isn't that AI replaced an engineering team outright. It's that AI collapsed the *coordination* cost that would normally require a team: a non-technical founder working with an unfamiliar developer would normally spend significant time as an unpaid translator — explaining product intent, interpreting vendor setup screens, judging which errors matter, chasing account access — the "Non-Technical CEO Burden" the CTO document names explicitly. Artifact discipline (a decision log, a living PRD, a risk register) converts that burden into durable objects a human can review quickly instead of holding in working memory continuously. That, more than raw code-generation speed, is the team's own stated explanation for why two people could carry a scope of work that would conventionally need a small team.

12.1 What artifact discipline doesn't substitute for: relational trust

There's a distinction worth drawing precisely, because the case study risks implying artifact discipline alone explains this: a **Founding CTO** role and a **CTO-advisory** engagement are not the same job, and the AI-assisted method described in this case study doesn't collapse the difference between them.

Advisory CTO work — a roadmap, a staffing plan, a moat/defensibility analysis, an investor deck — is work an outsider can do well without much relational history with the founder. It's evaluable on its own logic, largely independent of who's in the room. The CTO whose work this case study documents has done that kind of engagement for other founders separately, and it's genuinely a different job from the one documented here: those engagements are the easy part, by his own account.

The hard part, and the part this case study can't attribute to tooling, is what actually happens inside a Founding CTO relationship: grinding out code and policy decisions day to day with people you're still learning the strengths and weaknesses of, where consensus has to be built jointly rather than handed down. That's not a process artifact — the decision log, the changelog, and the risk register all capture *what* was decided and *why*, but they don't manufacture the trust required to reach a real decision together in the first place, especially when the "logical" analysis doesn't obviously win on its own (it doesn't always, and both people involved would say so).

This specific engagement is not a case of two strangers making that work through better tooling. The CTO and Founder had worked side by side for five years before this project began — the "red-team each other" dynamic documented in Section 5, and the fast, low-friction consensus visible throughout the decision log, sit on top of that prior relationship, not on top of the artifact-first process alone. A different Founding CTO search — one where the CEO and CTO are meeting for the first time — would face a real, harder problem

this case study's own evidence doesn't speak to: whether the same tooling and process discipline produces the same outcome absent that five years of established trust. The honest answer is this case study doesn't know, and shouldn't imply it does.

12.2 What the research says

This isn't just a plausible-sounding claim — there's real, peer-reviewed evidence directly on point, not the vendor-study caliber this case study has had to caveat elsewhere.

Supporting it directly: Greenberg and Mollick (Cornell/Wharton), "*The 'Devil' You Don't Know: A Test of the Detriments and Benefits of Co-founding with Strangers*," *Journal of Business Venturing*, 2025 — a real, named, peer-reviewed study, not a blog post. Its finding: founding teams with at least one member who was a stranger to the rest of the team before founding are **more than twice as likely to fail** as teams of friends, family, or prior coworkers. The stated mechanism is specific and relevant here: whatever advantage diverse strangers might bring in skills or background is outweighed by relational friction, and relational friction is a business-killer independent of technical competence. Separately, Harvard Business Review's July 2025 cofounder-matching research (drawing on Noam Wasserman's earlier work on founding-team conflict) makes the same point from the hiring-process side: people select cofounders by trust, and trust is easiest to find in already-familiar places — former coworkers, classmates, people already in one's orbit — because founding-team conflict is one of the primary documented reasons high-potential startups fail.

Checking the alternative hypothesis directly: is there evidence that structured process/documentation discipline (the artifact-first method this case study otherwise credits with a lot) can substitute for that prior trust, rather than merely complementing it? The organizational-psychology research on psychological safety and team performance says no, not as a substitute — psychological safety and structured process are consistently treated as **distinct factors**, not alternate paths to the same outcome, and reduced trust from limited in-person contact/history has an independent, negative effect on team performance regardless of how well-structured the process around it is. That's a real limit on this case study's own artifact-first-engineering thesis: the decision log, changelog, and risk register are evidence this project managed disagreement well, but the research doesn't support reading that management as something the artifacts caused on their own, absent the relationship they were layered onto.

What this doesn't settle: none of this research is about AI-agent-assisted development specifically, and none of it tests the counterfactual this case study can't test either — two people with no prior relationship, using this exact process, on this exact kind of build. The research supports the qualitative claim (prior relationship matters, a lot, and isn't something process discipline replaces) without validating any specific number attached to it here.

14. Build sequencing: how they knew where to start

The build order was not improvised. `00_CONTROL/PROJECT_INDEX.md` defines an explicit document reading order — canonical PRD, then the as-built annex, then the decision log, then schema and route-mapping specs, then the wireframes themselves — and the actual commit history follows that same dependency order in code: Next.js project scaffold → wireframe-derived route shells → authentication (Clerk) wired into the App Router → user/company sync via webhook → the first real external integration (Gmail OAuth, send-only) → Leads CRUD → Templates → Sequences → seeded demo data. This matches a principle stated directly in the PRD itself: build the data model and scheduling logic before adding deeper automation on top of it. Infrastructure and identity came first; product surface area came after there was a real account system to attach it to; the riskiest external integration (sending real email on a user's behalf) was proven early rather than last.

15. Honest state of completeness

The project maintains a self-audited functional matrix (`00_CONTROL/FUNCTIONAL_MATRIX.md`) that is candid about what's real versus placeholder, rather than letting a demo imply more than what's built. As of the most recent audit, all nine tracked application screens query real, tenant-scoped data — a milestone the matrix itself flags by name: real Gmail send and an unsubscribe footer, once "the single largest gap identified" in an internal MVP-readiness review, are now live and verified against the deployed environment. But the same document is equally explicit about what remains inert by design: six of seven outreach channels on the onboarding screen (SMS, LinkedIn, Instagram, Facebook, TikTok, Reddit) are visibly marked "(not built yet)" in the product itself, not silently hidden; billing is a placeholder; team-member management is an explicit "coming later." This kind of self-labeling — an unbuilt feature that says so in its own UI rather than pretending to work — is a small but telling detail about the team's discipline, and it's part of why the productivity claims in this case study are more credible than the typical anecdote: the same rigor that produced the time-log data also produced an honest account of what that time did and didn't buy.

16. What's compelling about this story beyond the outline

A few things emerged from the underlying documents that are worth naming as their own findings, not just supporting color:

The team measured itself, including its own tooling bugs. `build-time-log.ts` 's own history includes a self-caught measurement bug — an earlier version overstated real work time by roughly 6x by flooring every turn to a 15-minute minimum — fixed once the team noticed the numbers didn't match reality. A project willing to publicly correct its own measurement instrument, rather than quietly keep numbers that flattered the narrative, is unusual, and it's part of why the Section 10 figures are more trustworthy than the Section 4/10.4 estimates.

Real security bugs were found by the team's own process, not by an outside party. The cross-tenant admin-access bug (§5.2) is not a hypothetical risk named in a register — it's a real defect that existed in a shipped product and was caught by the team's own review discipline before it became an incident. That's a stronger claim about process quality than any productivity multiplier: the same speed that built the feature also caught the bug in it.

The compliance research (§5.3) is a second-order effect of AI-accelerated development that the team didn't originally set out to measure. Case-law-backed legal research at this depth, done by a two-person team without outside counsel, is itself evidence for the thesis — it's not just that code got written faster, but that *research and risk analysis* that would normally require hiring specialized help got compressed into the same loop.

The gap between June 14 and July 13 is data, not noise. A full month passed between the proof-of-concept session and the sustained build, with zero commits in between. That month wasn't a stall internal to this project — it was the CTO's time allocated elsewhere, confirmed directly — but it's still a useful corrective to any narrative implying continuous, uninterrupted AI-accelerated momentum from a single working method. Elapsed calendar time and measured project effort are two different axes, and this case study reports both rather than collapsing them into one continuous-looking timeline.

The no-AI-features decision (§12) is itself a case study in this document's own limits. A repository-only pass concluded the decision was undocumented and possibly made by default. Asking the CTO directly surfaced a specific, forward-looking scope call — AI features were deliberately parked as v2/post-GA work, pending real usage data on which feature would actually be compelling — plus a live, three-way hosting architecture question (frontier API vs. self-hosted open-weight models vs. a neutral compute-cloud provider) that included a genuine geopolitical sourcing concern about the origin of the strongest open-weight models. None of that made it into `DECISION_PATH.md`. That's a useful caution for anyone treating a project's own artifacts as a complete record: even a team this disciplined about writing decisions down has gaps — especially, it seems, around *why something is out of scope* rather than *why something is in scope* — and a case study built from documents alone, however thorough, is not the same as one that can still ask the people who made the call.

Appendix: Source Documents Referenced

- `99_ARCHIVE/predecessor_context/` — predecessor project persona and context
- `05_TRANSCRIPTS/Prototype Development Planning_transcript.txt` — the design walkthrough transcript
- `docs/DESIGN_WORKFLOW.md` — Figma/PRD/code ownership boundaries
- `docs/DEVELOPMENT_PROCESS_NOTES.md` — the June 14 session record and human-hour comparison table

- docs/CTO_DEVELOPMENT_APPROACH_ANALYSIS.md — the CTO's method self-analysis and productivity estimate
- 01_PRODUCT_REQUIREMENTS/Thalam_CANONICAL_PRD_v1.0.md and Thalam_AS_BUILT_ANNEX_v1.md — original spec and as-built divergence record
- 00_CONTROL/DECISION_PATH.md — append-only decision and reversal log
- docs/PLATFORM_PORTFOLIO_OPTIONS.md , docs/ARCHITECTURE_DIRECTION.md , docs/VERCEL_TO_GCP_MIGRATION.md — stack option analysis and portability posture
- CHANGELOG.md — dated flight-recorder of changes and reasoning
- docs/PROJECT_TRACKING.md — GitHub Issues/Project workflow definition
- scripts/session-close.ts , scripts/build-time-log.ts , scripts/generate-executive-report.ts — session automation
- 00_CONTROL/TIME_LOG_SCHEMA.md and 00_CONTROL/time_log.csv — the time-in-motion dataset underlying Section 10
- .github/workflows/ci.yml — CI pipeline
- 00_CONTROL/FUNCTIONAL_MATRIX.md — self-audited build-completeness record
- 00_CONTROL/RISK_REGISTER.md , 00_CONTROL/PRODUCTION_BUDGET_ESTIMATE.md — risk and cost posture
- 98_WORKING/article_notes/AI_ACCELERATED_DEVELOPMENT_ARTICLE_NOTES.md and ARTICLE_SESSION_BOOT.md — gitignored, untracked working notes the team kept for this exact article; source of the Gate 1/2 (aborted prior effort, founder invention) estimates and an earlier checkpoint of the Gate 6/7 execution figures, superseded here by the current time_log.csv totals
- Direct testimony from the CTO, gathered during the compilation of this document — the source for the corrected no-AI-features rationale (§12) and the seven-gate time model (§2, §10)
- 2026 market-rate research (contractrates.fyi, IT-consulting rate guides, offshore-rate-by-country reports) — source for the domestic/offshore rate validation in §10
- Gartner, Deloitte's 2024 Global Outsourcing Survey, and a peer-reviewed 2026 offshore-quality study (Oxford-archived) — source for the offshore rework-risk figures in §10
- METR's July 2025 RCT, Stanford (Perry et al.) on AI-assisted code security, and vendor studies (CodeRabbit, Veracode, GitClear) — source for the AI-coding-failure-mode research in §11